Report on CSC3050 Project 1

The report consists of the six logic codes: add.s, add_extra.s, and.s, beq.s, required.s, sll.s., as well as their result and implementation. Also, the naïve function that needs to be optimized is also concluded, as well as its runtime.

Part I: implementation of logics

This part is relatively easy. The implementation of and.s is included in the project manual, so I just followed the way and completed all the works. The pictures below shows my implementation and their result.

Add.s:

```
ADD:
    # please start your code
    lui a3,%hi(registers)
    addi a3,a3,%lo(registers)
    slli a0,a0,2
    slli a1,a1,2
    slli a2,a2,2
    add a0, a3, a0
    add a1, a3, a1
    add a2, a3, a2
    lw a0,0(a0)
    lw a1,0(a1)
    add a1,a0,a1
    sw a1,0(a2)
    li a0, 0
    jr ra
    .size   ADD, .-ADD
```

```
student@b4dd90616e8c:~/Assignment1/finished$ qemu-riscv32 add
Test 1: ADD
Pass
Test 2: ADD
Pass
Test 3: ADD
Pass
Do not include overflow detection: ADD
You haven't included it, good job!!!
student@b4dd90616e8c:~/Assignment1/finished$
```

Add_extra.s

Here, to detect overflow, I implemented a method to look at the first bit of both added numbers. If two positive numbers result in a negative number, then there is a overflow and vice versa.

add_extra.s

Assignment1 > finished > add_extra.s

```
18  ADD:
20      lui a3,%hi(registers)
21      addi a3,a3,%lo(registers)
22      slli a0,a0,2
23      slli a1,a1,2
24      slli a2,a2,2
25      add a0,a3,a0
26      add a1,a3,a1
27      add a2,a3,a2
28      lw t0,0(a0)
29      lw t1,0(a1)
30      add t2,t0,t1
31      sw t2,0(a2)
32      xor t3,t0,t1
33      srai t3,t3,31
34      not t3,t3
35      xor t4,t0,t2
36      srai t4,t4,31
37      and a0,t3,t4
38      snez a0,a0
39      jr ra
```

```
student@b4dd90616e8c:~/Assignment1/finished$ qemu-riscv32 add_extra
Test 1 Overflow: ADD
Pass
Test 2 Overflow: ADD
Pass
Test 3 Overflow: ADD
Pass
End of test
student@b4dd90616e8c:~/Assignment1/finished$
```

And.s

```
18  AND:
19      # please start your code
20      lui a3,%hi(registers)
21      addi a3,a3,%lo(registers)
22      slli a0,a0,2
23      slli a1,a1,2
24      slli a2,a2,2
25      add a0, a3, a0
26      add a1, a3, a1
27      add a2, a3, a2
28      lw a0,0(a0)
29      lw a1,0(a1)
30      and a1,a0,a1
31      sw a1,0(a2)
32      li a0, 0
33      jr ra
34      .size   AND, .-AND
```

```
student@b4dd90616e8c:~/Assignment1/finished$ q
emu-riscv32 and
Test 1: AND
Pass
Test 2: AND
Pass
Test 3: AND
Pass
End of test
student@b4dd90616e8c:~/Assignment1/finished$
```

Beq.s: return 1 if equal, return 0 if not equal

```
17      .type   BEQ, @function
18  BEQ:
19      # please start your code
20      lui a3, %hi(registers)
21      addi a3, a3, %lo(registers)
22      slli a0, a0, 2
23      slli a1, a1, 2
24      add a0, a3, a0
25      add a1, a3, a1
26      lw a0, 0(a0)
27      lw a1, 0(a1)
28      beq a0, a1, equal
29      li a0, 0
30      jr ra
31  equal:
32      li a0, 1
33      jr ra
34      .size   BEQ, .-BEQ
```

```
student@b4dd90616e8c:~/Assignment1/finished$ q
emu-riscv32 beq
Test 1: BEQ
Pass
Test 2: BEQ
Pass
Test 3: BEQ
Pass
End of test
student@b4dd90616e8c:~/Assignment1/finished$
```

Required.s

```
NOT:
    # please start your code
    lui a3,%hi(registers)
    addi a3,a3,%lo(registers)
    slli a0,a0,2
    slli a1,a1,2
    add a0, a3, a0
    add a1, a3, a1
    lw a0,0(a0)
    not a0,a0
    sw a0,0(a1)
    li a0,0
    jr ra
    .size   NOT, .-NOT
    .align  2
```

```
SUBI:
    # please start your code
    lui a3,%hi(registers)
    addi a3,a3,%lo(registers)
    slli a0,a0,2
    slli a2,a2,2
    add a0, a3, a0
    add a2, a3, a2
    lw t0,0(a0)
    sub t0,t0,a1
    sw t0,0(a2)
    li a0,0
    jr ra
    .size   SUBI, .-SUBI
```

```
BG:
    # please start your code
    lui a3,%hi(registers)
    addi a3,a3,%lo(registers)
    slli a0,a0,2
    slli a1,a1,2
    add a0, a3, a0
    add a1, a3, a1
    lw t0,0(a0)
    lw t1,0(a1)
    slt t2,t1,t0
    sw t2,8(a3)
    lw a0,8(a3)
    jr ra
    .size   BG, .-BG
```

```
student@b4dd90616e8c:~/Assignment1/finished$
emu-riscv32 required
Test 1: NOT
Pass
Test 2: NOT
Pass
Test 3: NOT
Pass
End of NOT test
Test 1: SUBI
Pass
Test 2: SUBI
Pass
Test 3: SUBI
Pass
End of SUBI test
Test 1: BG
Pass
Test 2: BG
Pass
Test 3: BG
Pass
End of BG test
student@b4dd90616e8c:~/Assignment1/finished$
```

sll.s

```
SLL:
    # please start your code
    lui a3,%hi(registers)
    addi a3,a3,%lo(registers)
    slli a0,a0,2
    slli a1,a1,2
    slli a2,a2,2
    add a0, a3, a0
    add a1, a3, a1
    add a2, a3, a2
    lw t0,0(a0)
    lw t1,0(a1)
    li t1, 2
    sll t1,t0,t1
    sw t1,0(a2)
    li a0, 0
    jr ra
    .size   SLL, .-SLL
```

```
student@b4dd90616e8c:~/Assignment1/finished$
emu-riscv32 sll
Test 1: SLL
Pass
Test 2: SLL
Pass
Test 3: SLL
Pass
End of test
student@b4dd90616e8c:~/Assignment1/finished$
```

Therefore, all implementations are done.

Part II: Optimizing a program

This program is about printing prime numbers from 2 to 100000. I first checked its original time and the time after extending the range to 1000000, the result is shown below.

```
问题  输出  终端  ···  bash - finished
7 99823 99829 99833 99839 99859 99871 99877 99
881 99901 99907 99923 99929 99961 99971 99989
99991
real    0m1.029s
user    0m1.055s
sys     0m0.010s
student@b4dd90616e8c:~/Assignment1/finished$
```

```
问题  输出  终端  ···  bash - finished
683 999721 999727 999749 999763 999769 999773
999809 999853 999863 999883 999907 999917 9999
31 999953 999959 999961 999979 999983
real    1m23.719s
user    1m23.323s
sys     0m0.041s
student@b4dd90616e8c:~/Assignment1/finished$
```

The range extension code is shown below. (Only line 63 is modified.)

```
58  main:
59      addi    sp,sp,-32
60      sw  ra,28(sp)
61      sw  s0,24(sp)
62      addi    s0,sp,32
63      li  a5,98304
64      addi    a5,a5,1696
```

```
58  main:
59      addi    sp,sp,-32
60      sw  ra,28(sp)
61      sw  s0,24(sp)
62      addi    s0,sp,32
63      li  a5,998304
64      addi    a5,a5,1696
```

It is obvious that the program runs a long time. To reduce runtime, I added four lines in the .L5 part, which is originally used to add one on the dividing number during the prime number test. When the dividing number reaches the square root of test number, the test number is automatically considered a prime number, which reduces the time complexity from O(n^2) to O(n^1.5) and greatly saves time. The code I added is shown below.

```
31  .L5:
32      lw  a5,-20(s0)
33      addi    a5,a5,1
34      sw  a5,-20(s0)
35      lw  t0,-20(s0)  #put i into t0
36      lw  t1,-36(s0)  #put n into t1
37      mul t0,t0,t0    #t0 = i*i
38      bgt t0,t1,.L3   #if i*i>n, end loop
```

After adding the codes, I re-tested the runtime, and it turned out that the runtime is significantly reduced.

```
问题  输出  终端  ···  bash - finished
7 99823 99829 99833 99839 99859 99871 99877 99
881 99901 99907 99923 99929 99961 99971 99989
99991
real    0m0.029s
user    0m0.018s
sys     0m0.000s
student@b4dd90616e8c:~/Assignment1/finished$
```

```
问题  输出  终端  ···  bash - finished
683 999721 999727 999749 999763 999769 999773
999809 999853 999863 999883 999907 999917 9999
31 999953 999959 999961 999979 999983
real    0m0.362s
user    0m0.270s
sys     0m0.010s
student@b4dd90616e8c:~/Assignment1/finished$
```

I also tested a bigger range. The previous time used for 1000000 range can now be used for 50000000, which is a 50 times improvement.

```
终端  ···  bash - finished  ···
01 49999819 49999843 49999847 49999853 4999987
7 49999883 49999897 49999903 49999921 49999991

real    0m56.797s
user    0m55.544s
sys     0m0.757s
student@b4dd90616e8c:~/Assignment1/finished$
```